

# Opus

## SUMMARY

---

For decades, I have worked with software, databases, enterprise systems, project management, finance, architecture, and the difficult problem of translating human intentions into systems that actually work.

Eventually, I arrived at a conclusion.

The real problem is rarely software.

The real problem is the distance between what we need, what we understand, what we design, and what we finally build.

OPUS is my attempt to reduce that distance.

It has gradually become much larger than a software project.

## START WITH REALITY

---

Every serious project begins with something that exists before requirements, specifications, software or plans.

I call this x.

x is the situation we are trying to understand or change.

ZenOps expresses the transformation approximately as:

$x \rightarrow m(x) = (\text{objects, relations}) \rightarrow \text{understanding} \rightarrow \text{pattern} \rightarrow \text{implementation} \rightarrow \text{evidence}$

## FROM CHAOS TO STRUCTURE

---

Human beings experience reality as an enormous mixture of events, people, things, intentions, constraints and relationships.

Yet underneath much of that complexity is a surprisingly simple structure:

Objects and relations.

## NEED BEFORE REQUIREMENT

---

Traditional development frequently speaks about requirements.

I believe there is something before a requirement:

Need.

That distinction led to the Need Definition Document, or NDD.

A complex problem can gradually be decomposed:

Need  $\rightarrow$  sub-need  $\rightarrow$  sub-need  $\rightarrow$  actionable element

## FLEXI: WORK IN SMALL EXECUTABLE UNITS

---

After QT, work can move into short execution cycles.

I call this approach FLEXI.

A useful cycle becomes:

Need  $\rightarrow$  implementation  $\rightarrow$  test  $\rightarrow$  evidence  $\rightarrow$  learning

## OPUS DELIVERY

---

OPUS Delivery emerged as the software environment for this way of working.

The ambition is to connect the entire chain:

Experience → Explication → ORIGIN → Pattern → Meta-Model → Delivery → Implementation → Evidence

## **OPUS.NET**

---

From this thinking came OPUS.NET.

The goal of OPUS.NET is a reusable distributed application infrastructure.

CRUDME: Create Read Update Delete Method Event

## **THE OBJECT NETWORK**

---

In OPUS.NET identity can be represented by an OPUSGuid.

The persisted representation can then become conceptually minimal:

GUID + serialized object

## **A BINARY INTERNET**

---

For tightly controlled OPUS applications, a deterministic binary TCP/IP protocol can be smaller and conceptually closer to the domain architecture.

## **ONE INFRASTRUCTURE, MANY PRODUCTS**

---

ENTER explores digital entertainment distribution.

GameX explores an MMORPG-like artificial society.

MEL explores streamlined direct marketing operations.

OPUS Delivery explores project execution and knowledge transformation.

OPUS Designer can explore visual construction of systems.

OPUS ERP can apply the same infrastructure to enterprise management.

Black Balder can apply it to trading.

Alina can connect artificial intelligence to structured object networks.

## **ENTER AND THE ECONOMICS OF DIGITAL OBJECTS**

---

Instead of thinking primarily in terms of web pages, think in terms of content objects.

## **GAMEX: SOFTWARE AS SOCIETY**

---

The player participates in an evolving artificial society.

The game becomes entertainment, economic simulation and education simultaneously.

## **EDUCATION**

---

The future increasingly rewards the ability to construct useful models from unfamiliar situations.

## **PATTERN THINKING**

---

A pattern can therefore become a reusable unit of knowledge.

The long-term possibility is a pattern library spanning domains.

## **LONGEVITY**

---

Longevity research belongs within the broader picture as another complex system and object-and-relation problem.

## **ARTIFICIAL INTELLIGENCE CHANGES THE EQUATION**

---

Natural language changes the interface between human intention and computing.

The distance between thought and executable systems begins to shrink.

## **SOFTWARE IS NOT THE DESTINATION**

---

Software is leverage.

Software therefore converts patterns of thought into operational systems.

## **THE LARGER OPUS IDEA**

---

Reality → Observation → Need → Objects and Relations → Patterns → Models → Software → Distribution → Execution → Evidence → Learning

## **THE FUTURE**

---

Can we build better systems by first learning to model reality better?

what exists → what should exist → evidence that we actually made it work.